EX25 · Vectores, sistemas lineales y precisión

Fundamentos físicos, desarrollo matemático, ejemplo manual, condicionamiento e interpretación.

LANGUAGE / ID	UNIT	HANDOUT
ES · EX25	algebra / vectors and linear systems	Guía de teoría matemática e ingeniería

1 · ¿Qué problema de ingeniería resolvemos?

Supón que dos actuadores empujan una pequeña plataforma en un plano. Las direcciones son conocidas, pero queremos encontrar las magnitudes que producen una fuerza resultante concreta. Este problema aparece en control de vehículos, reparto de cargas, mecanismos y calibración de sensores.

Para una fuerza de módulo F y dirección unitaria $d=(d_x,d_y)$, la fuerza vectorial es Fd. Al sumar las contribuciones de ambos actuadores obtenemos el equilibrio de fuerzas. Ignoramos momentos, límites máximos y dinámica: es un modelo **lineal y estático**.

$$\mathbf{F} = u_1\hat{\mathbf{d}}_1 + u_2\hat{\mathbf{d}}_2$$

2 · Del modelo físico a una matriz

Cada fuerza tiene componentes horizontal x y vertical y. Las direcciones unitarias son $d_1=(0{,}8;0{,}6)$ y $d_2=(0{,}6;-0{,}8)$. Queremos que la suma sea b=(14;2) N.

```
Fx = 0.8*u1 + 0.6*u2 = 14 N
Fy = 0.6*u1 - 0.8*u2 =  2 N

         |  0.8   0.6 |   |u1|   |14|
A @ u =  |  0.6  -0.8 | @ |u2| = | 2|
```

Cada **columna** de A corresponde a un actuador; cada **fila** corresponde a una componente de fuerza. Si añadimos actuadores, la matriz gana columnas; si añadimos direcciones de equilibrio, gana filas.

3 · Recordatorio de vectores: módulo y producto escalar

$$\|\mathbf{v}\|_2 = \sqrt{v_x^2 + v_y^2}$$

El módulo expresa la longitud del vector. En fuerzas, tiene unidades de newtons. Para la dirección d_1: $\sqrt{(0{,}8^2+0{,}6^2)}=\sqrt{1}=1$, de modo que se trata de una dirección unitaria.

$$\mathbf{a}\cdot\mathbf{b} = a_x b_x + a_y b_y$$

El producto escalar entre direcciones unitarias permite comprobar si son ortogonales: $0{,}8\cdot0{,}6+0{,}6\cdot(-0{,}8)=0$. Las dos direcciones del ejercicio forman 90 grados. Esta geometría tiene una consecuencia muy favorable en la estabilidad numérica.

4 · Ejemplo resuelto a mano: sistema 2×2

Antes de programar, resuelve el siguiente sistema más simple. Queremos encontrar x e y:

```
3*x + 1*y = 9
1*x + 2*y = 8
```

Por sustitución: de la segunda ecuación, x=8−2y. Sustituye en la primera: 3(8−2y)+y=9, de donde 24−5y=9, así que y=3 y x=2.

Verificación: 3·2+1·3=9; 1·2+2·3=8. Por tanto, x=(2,3) es solución. El software debe poder llegar al mismo resultado usando un método numérico de álgebra lineal.

5 · Determinante e invertibilidad

Para una matriz de tamaño 2×2, A=[[a,b],[c,d]], el determinante es ad−bc. Cuando vale cero, las ecuaciones no definen una solución única: el sistema es **singular**.

$$\det(\mathbf{A}) = ad - bc$$

En el ejemplo manual, det(A)=3·2−1·1=5 ≠ 0. En nuestro sistema de actuadores, det(A)=0,8·(−0,8)−0,6·0,6=−1. Por eso existe una solución única.

Es mejor resolver mediante factorizaciones numéricas (como hace **np.linalg.solve**) que calcular explícitamente A^{-1}. Resolver un sistema y obtener una inversa son tareas matemáticamente relacionadas, pero no computacionalmente idénticas.

6 · Resultado manual del caso físico

Como las columnas de A son ortonormales, $A^TA=I$. Multiplicando ambos lados de Au=b por A^T obtenemos $u=A^Tb$. Esta simplificación particular no vale para cualquier matriz.

$$\mathbf{A}^T\mathbf{A} = \mathbf{I}$$
$$\mathbf{u} = \mathbf{A}^T\mathbf{b}$$

```
u1 = 0.8*14 + 0.6*2 = 12.4 N
u2 = 0.6*14 - 0.8*2 =  6.8 N

Fx = 0.8*12.4 + 0.6*6.8 = 14 N
Fy = 0.6*12.4 - 0.8*6.8 =  2 N
```

Ambas magnitudes resultan positivas. En el modelo elegido son fuerzas físicamente compatibles con actuadores que solo empujan. Eso no implica que lo sigan siendo para otro objetivo; una solución lineal puede contener fuerzas negativas o superar los límites reales.

7 · Error de cálculo frente a error de modelado

El **residual** representa el error al reconstruir el objetivo con nuestra solución numérica. Si r=Au−b, el residual absoluto es $||r||_2$ (N) y el relativo es $||r||_2/||b||_2$ (sin unidades, si b≠0).

$$\rho = \frac{\|\mathbf{Au} - \mathbf{b}\|_2}{\|\mathbf{b}\|_2}$$

Un residual pequeño indica consistencia interna entre datos, modelo y solución computada. **No demuestra** que el modelo físico incluya todos los efectos reales (momentos, rozamiento, incertidumbre, saturación o errores de calibración). Tampoco garantiza que una solución sea insensible a errores en los datos.

8 · Condicionamiento y amplificación de perturbaciones

Imagina medir b con un pequeño error δb. Al resolver A(u+δu)=b+δb, el cambio en la solución es δu. Para matrices invertibles, la relación de cambios relativos está acotada por el número de condición en norma 2.

$$\frac{\|\delta \mathbf{u}\|_2}{\|\mathbf{u}\|_2} \leq \kappa_2(\mathbf{A}) \frac{\|\delta \mathbf{b}\|_2}{\|\mathbf{b}\|_2}$$

$\kappa_2(A) \geq 1$ para matrices invertibles. Cuanto mayor sea, mayor puede ser la sensibilidad. Para nuestro sistema, la matriz ortogonal tiene $\kappa_2(A)=1$: no amplifica los cambios relativos del objetivo.

```
delta_b = [0.000, 0.010] N
u_original  = [12.400, 6.800] N
u_perturbed = [12.406, 6.792] N
delta_u     = [ 0.006,-0.008] N

||delta_b|| = 0.010 N
||delta_u|| = 0.010 N
relative change ~ 0.07071% in both.
```

9 · Cuando las ecuaciones casi son iguales

Considera A=[[1,1],[1,1,000001]], u=(10,5) y b=(15,15,000005). Si alteramos únicamente la segunda componente de b en una millonésima, la nueva solución es aproximadamente (9,6). ¡Una perturbación diminuta puede generar un cambio de orden uno!

```
A_bad = [[1, 1], [1, 1.000001]]
u       = [10, 5]
b       = [15, 15.000005]
new_b   = [15, 15.000006]
new_u   = [9, 6]

cond_2(A_bad) ~ 4,000,002
```

Aquí ambas ecuaciones aportan información casi redundante. El determinante vale 10^{-6} y el condicionamiento es muy desfavorable. El residual puede seguir siendo minúsculo, pero inferir u con precisión exige datos de entrada extremadamente fiables.

10 · Aritmética de punto flotante

Python/NumPy suelen representar reales con **float64**: precisión aproximada de 15–16 cifras decimales significativas, no números reales exactos. Sumas, restas y productos redondean a valores binarios representables.

```
0.1 + 0.2 == 0.3    # False
np.isclose(0.1 + 0.2, 0.3)  # True
```

Diferencia tres conceptos: **error de redondeo** (representación y operaciones), **error de truncamiento** (aproximaciones de un método, por ejemplo Euler) y **error de modelado** (física idealizada). Un buen algoritmo numérico debe evaluar cada uno por separado.

11 · Cómo interpretar los resultados

Resultado	Interpretación en ingeniería
u1, u2 [N]	Fuerza que solicita el controlador a cada actuador
Au [N]	Fuerza realmente reconstruida en el modelo
Residual absoluto [N]	Diferencia de fuerzas en unidades físicas
Residual relativo [–]	Diferencia normalizada por el objetivo
$\kappa_2(A)$ [–]	Sensibilidad intrínseca de la geometría del sistema
Cambio relativo [–]	Efecto de una perturbación concreta en el objetivo

12 · Resumen y conexión con el código

- La **matriz** es un modelo geométrico y físico, no solo una tabla de números.
- **solve_forces** transforma objetivos de fuerza en comandos para actuadores.
- **residual_norm** y **relative_residual** verifican coherencia numérica.
- **sensitivity_analysis** distingue precisión del cálculo y robustez frente a datos perturbados.
- Los **tests** comprueban resultados, formas y errores antes de confiar en una implementación.

> **Idea central:** un buen cálculo de ingeniería no termina al obtener números. Debemos justificar el modelo, validar las unidades y cuantificar cuánto podemos confiar en el resultado.